

Serial Decode — Keithley DMM6500, DAQ6510, DMM7510, SMU2461

A UART decoder that runs **on** a Keithley bench instrument. It digitizes the line with the instrument's own digitizer, recovers the baud rate, frame format and idle polarity from the signal, and shows the bytes on the front panel as text or hex. No host, no logic analyser, one probe.

Ian Ameline · **version 1.32** · MIT licence (see [LICENSE](#))

SERIAL DECODE - 240B FRAME

End App

BAUD	FORMAT	IDLE	LOGIC	THRESH	SA/BIT	RATE	BYTES	ERR	FIT	S/N
9600	8N1	HIGH	3V3 CMOS	1.63 V	8.3	80 kS/s	239	0	1.00	74 dB
Start bit										
0000	8B 81 69 6E 20 63 75 6C					70 61 20 71 75 69 20 0D		??in culpa qui .		
0016	0A 6F 66 66 69 63 69 61					20 64 65 73 65 72 75 6E		.officia deserun		
0032	74 20 6D 6F 6C 6C 69 74					20 61 6E 69 6D 20 69 64		t mollit anim id		
0048	20 65 73 74 20 6C 61 62					6F 72 75 6D 2E 20 4C 6F		est laborum. Lo		
0064	72 65 6D 20 69 70 73 75					6D 20 64 6F 6C 6F 72 20		rem ipsum dolor		
0080	73 69 74 20 0D 0A 61 6D					65 74 2C 20 63 6F 6E 73		sit ..amet, cons		
0096	65 63 74 65 74 75 72 20					61 64 69 70 69 73 63 69		ectetur adipisci		
0112	6E 67 20 65 6C 69 74 2C					20 73 65 64 20 64 6F 20		ng elit, sed do		
0128	65 69 75 73 6D 6F 64 20					74 65 6D 70 6F 72 20 69		eiusmod tempor i		
0144	6E 63 69 64 69 64 75 6E					74 20 0D 0A 75 74 20 6C		ncididunt ..ut l		
0160	61 62 6F 72 65 20 65 74					20 64 6F 6C 6F 72 65 20		abore et dolore		
0176	6D 61 67 6E 61 20 61 6C					69 71 75 61 2E 20 55 74		magna aliqua. Ut		
0192	20 65 6E 69 6D 20 61 64					20 6D 69 6E 69 6D 20 76		enim ad minim v		
0208	65 6E 69 61 6D 2C 20 71					75 69 73 20 0D 0A 6E 6F		eniam, quis ..no		
0224	73 74 72 75 64 20 65 78					65 72 63 69 74 61 74		strud exercitat		
FRAME HEX pg 1/1 bytes 1-239/239 win 240 [done]						log: bytes053 719 B				
CaptureViewModeNewLogSaveOptions										

The instrument's own front panel, no host attached: 239 bytes of a 240-byte capture at 9600 baud 8N1, no framing errors, S/N 74 dB. The padlock on the BAUD cell means the rate is pinned here rather than re-detected each capture.

docs/MANUAL.md	using it: hooking up, the screen, the buttons, what it copes with, where it fails
docs/REFERENCE.md	every measured number, and the firmware limits behind them
docs/BENCH.md	the harness: the seeded sweep, the release gate stage by stage, reproducing a failure

Ships as `Serial_Decode.tspa`, installed from a USB key through the instrument's own **Manage Apps** screen. Written in TSP — Lua **5.0.2** embedded in the firmware, so the sources avoid `#`, `%`, `string.gmatch` and bitwise operators throughout.

Installing it

The app ships as a single file, `Serial_Decode.tspa`. It installs from a USB key through the instrument's own **Manage Apps** screen — no host software, no driver, nothing to compile. The archive is plain ASCII: a `loadscript Serial_Decode` wrapper around the six modules and an icon, so it can be read and diffed like any other source file.

1. **Copy the archive to a USB key.** Put `Serial_Decode.tspa` in the **root** of a FAT-formatted key. Leave the name as it is.
2. **Put the key in the front-panel USB port.**
3. **Install it from the front panel,** through **MENU ▶ Manage Apps**, which copies the app into the instrument's internal memory. The exact wording of that screen varies between models and firmware revisions; it is the one that lists the apps on the key.
4. **Run it** from the instrument's **Apps** menu. The app takes its first capture as it starts, so a line that is already carrying traffic decodes immediately — and if nothing is connected yet it says `line is idle (no transitions)` and waits.
5. **Leave the key in, or take it out.** With a key in the slot the app appends decoded bytes to a log and the `Save` button writes a report; with no key it captures and decodes exactly the same in every mode, says `log: no USB key`, and reports a recording as `NOT recorded` rather than naming a file. The key may be removed and replaced while the app runs.

Without installing anything

For development, the modules load straight over the LAN, which is the same code the archive carries:

```
python3 tools/run_app.py          # loads tsp/*.tsp over the socket and calls
sdec.start()
```

One caveat the tool states itself: the front panel can be built **once per power cycle**, so a second load needs a clean exit first or the instrument has to be power-cycled.

Hooking up the probe

INPUT HI and **INPUT LO** on the front terminals. **LO** to the ground of the circuit you are probing, **HI** to the line you want to read — everything is measured at HI relative to LO, so without a shared ground the readings mean nothing.

- **Keep the line between –10 V and +10 V.** The app fixes the meter on its 10 V range. 3.3 V CMOS, 5 V TTL and a 6 V LIN line tapped to ground need nothing in front of them. For 12 V, use a 2:1 divider — and on an open-drain bus pick resistors big enough not to load it down.
- **A DC offset does not matter.** The app measures the two levels on your line and puts its threshold between them, and it works out the idle polarity from the signal itself.
- **Do not put a differential bus across HI and LO.** RS-485, CAN and LIN want a transceiver first, or tap one side of the pair against ground.
- **A signal whose band straddles ground is decoded inverted, deliberately** — see [docs/BETA.md](#), which lists the behaviours that look like defects and are not.

Then: press **Capture** while the device is talking, and **View** to switch between hex and text. The [manual](#) covers the rest of the panel, the bit rates it reaches, and what a marginal signal looks like.

Building the archive yourself

Only needed if you have changed anything in `tsp/`:

```
python3 tools/package_tspa.py      # concatenates tsp/*.tsp into Serial_Decode.tspa
```

Everything else

docs/PROJECT.md	what to know before trusting it, the measured endurance figures, which instruments it runs on, how to report a bug, and the release gates
tools/README.md	the harness: offline suites, the bench sweep, the release gate